



YAML

person1, 35, pune, 50000 → raw data / facts → meaningless

↓
formatting (serialization) → XML / JSON / YAML

↓
information → meaningful

tag
↓
XML
content / data
↓
< person >
 < name > person1 < / name >
 < address > pune < / address >
 < age > 35 < / age >
 < salary > 50000 < / salary >
< / person >

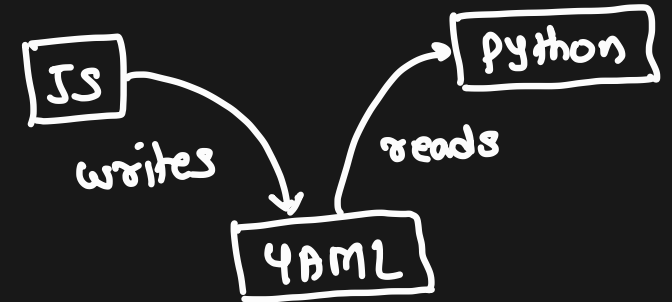
JSON
{
 "name": "person1",
 "address": "pune",
 "age": 35,
 "salary": 50000
}

YAML
name: person1
age: 35
address: pune
salary: 50000

Overview



- YAML is the abbreviated form of “YAML Ain’t Markup Language”
- Human-readable data serialization standard
- Commonly used for configuration files (Ansible, Kubernetes, Docker Compose, CI/CD pipelines)
- Aim: Easy for humans to read/write, while being portable between programming languages
- **Basic Syntax Rules**
 - Case-sensitive
 - Whitespace-sensitive (indentation defines structure – use spaces, not tabs)
 - Comments start with # (only single-line comments)
 - File extensions: .yaml or .yml



Features



- Matches native data structures of agile methodology and its languages such as Perl, Python, PHP, Ruby and JavaScript
- YAML data is portable between programming languages
- Includes data consistent data model
- Easily readable by humans
- Supports one-direction processing
- Ease of implementation and usage

Basics



- YAML is case sensitive
- The files should have .yaml or .yml as the extension
- YAML does not allow the use of tabs while creating YAML files; spaces are allowed instead ❄
- Comment starts with #
- Comments must be separated from other tokens by whitespaces

#comment here ❌

comment here ✓
↓
space is mandatory

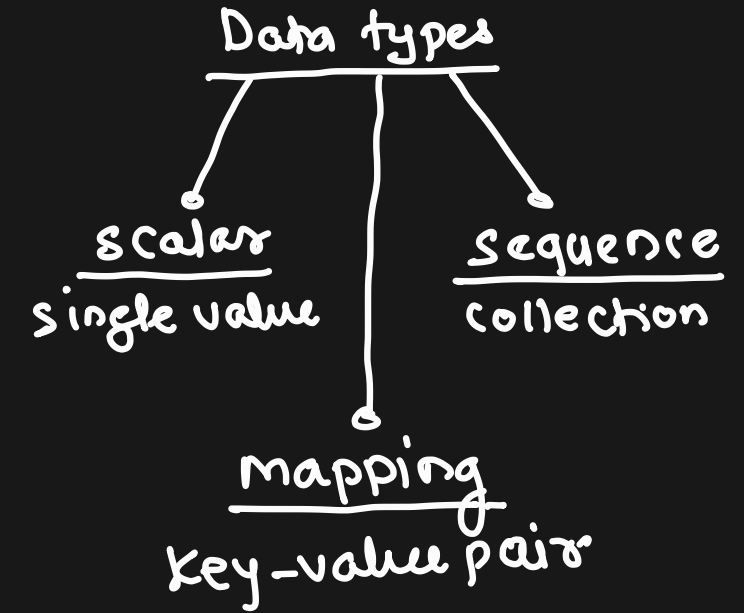
Common Problems



- Indentation: Use spaces (usually 2). Inconsistent spaces cause errors
- No tabs – YAML parser will fail
- Quotes needed for strings that start with @, :, &, *, %, #, !, |, >
- Booleans: yes/no or on/off may be parsed as booleans – use quotes if you want literal strings: "yes"

Scalars

- Scalars in YAML are written in block format using a literal type
- E.g.
 - Integer
 - 20
 - 40
 - String
 - Steve
 - "Jobs"
 - 'USA'
 - Float
 - 4.5
 - 1.23015e+3



Mapping



- Represents key-value pair
- The value can be identified by using unique key
- Key and value are separated by using colon (:)
- E.g.

name: person1

address: "India"

phone: +9145434345

age: 40

hobbies:

- reading

- playing

Sequence



- Represents list of values
- Must be written on separate lines using dash and space
- Please note that space after dash is mandatory
- E.g.

pet animals

- cat

- dog

programming languages

- C

- C++

- Java

Sequence

- Sequence may contains complex objects

E.g. Key

- products:

- **title**: product 1

- ① **price**: 100

- description**: good product

- **title**: product 2

- ② **price**: 300

- description**: useful product

→ value
sequence

```
{ "products": [ { "title": "product1",  
                  "price": 100,  
                  ...  
                },  
                { "title": "product2",  
                  "price": 200,  
                  ...  
                }  
              ]  
}
```